



STM32F4 四轴飞行器开发手册

硬件与软件部分

摘要

本文档对四轴飞行器的硬件设计、软件框架作讲解与介绍，助力于读者能快速理解飞行器的软件结构和所有功能的实现原理。文档内容不包括四轴飞行器的控制原理与算法部分，该部分内容请查看资料内《四轴无人机开发手册 - 原理与算法部分》文档。

光电 209 实验室

目录

1.软件资源概览	2
2.软件框架设计说明	3
2.1 默认任务	3
2.2 显示任务	3
2.3 传感器数据处理任务	4
2.4 平衡控制任务	4
2.5APP、PS2 控制任务	5
2.6 雷达数据处理任务	6
2.7 光流数据处理中断	6
3.硬件与软件实现细节	7
3.1 陀螺仪	7
1 陀螺仪电路设计	7
2 陀螺仪数据获取	7
3.2 激光测距	8
3.3 电量检测	8
1 电压测量电路实现	9
2 程序实现	9
3.4 电机控制	9
1 硬件部分	9
3.5PS2 手柄控制	12
1 硬件部分	12
2 软件部分	13
3.6APP 控制	15
1 硬件部分	15
2 软件部分	15

1.软件资源概览

本产品使用到的主控芯片为 **STM32F405RGT6**，使用到的资源以及作用如下介绍：

ADC1_IN1: ADC1 通道 1，用于采集电池的输入电压，监测电池电量

TIM6: 计数定时器，用于 FreeRTOS 系统调试

TIM8: 定时器 8 所有通道，PWM 输出，用于驱动电调控制电机

TIM7: HAL 库时基

SysTick: FreeRTOS 系统时基

I2C1: 硬件 IIC1，用于与陀螺仪通信，获取陀螺仪数据

USART1: 调试串口，同时也是程序烧录口

USART2: 串口 2，与雷达模块通信

USART3: 串口 3，与光流模块通信，获取光流传感器数据

USART4: 串口 4，与蓝牙模块通信，用于手机 APP 控制与查看波形功能

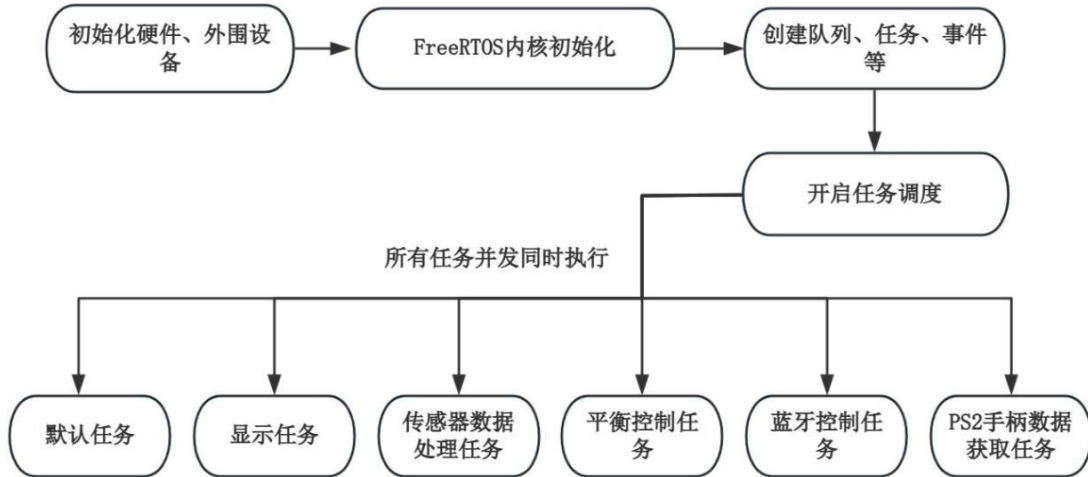
UART5: 串口 5，与激光测距模块 STP23L 通信，获取高度信息

USB_OTG_FS: 全速 USB 接口，用于获取 PS2 手柄数据

GPIO: 普通 GPIO,用于 LED、蜂鸣器、OLED 等功能。其中 OLED 为预留接口。

2.软件框架设计说明

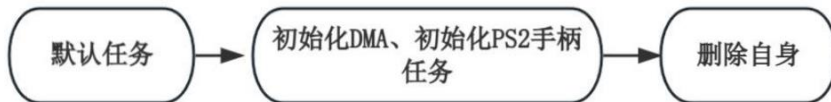
为保证四轴飞行器运行的实时性以及 CPU 资料的合理分配使用，整体软件框架基于 FreeRTOS 设计，运行框架如图所示：



对于具体的任务优先级配置，可查看文件 `freertos.c` 中的 `MX_FREERTOS_Init()` 函数。在任务创建并开始调度后，每个任务将会同时运行。

2.1 默认任务

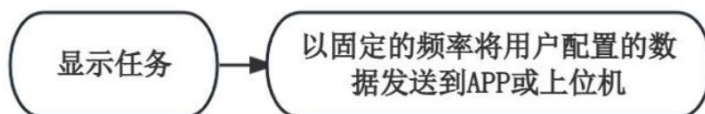
在默认任务内，会创建 USB 手柄接收任务，并使能各个传感器的 DMA 接收，以便实时获得数据。在默认任务完成后，会删除自身以节省资源空间。任务名称为 `voidStartDefaultTask(void*argument)`。整体运行逻辑如下图所示。



2.2 显示任务

显示任务主要负责将飞行器的实时状态进行上传。可以通过程序来选择上传到手机 APP 或 `minibalance` 上位机。上传的数据包括电池电压、欧拉角、飞行高度等数据，具体内容可以在程序中进行自定义调节。显示任务函数名为 `ShowTask(void*param)`。

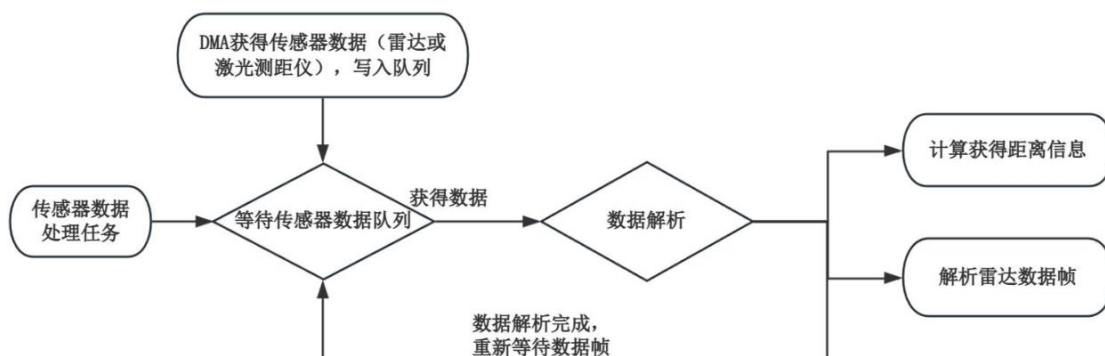
由于显示任务为调试属性，所以将其优先级配置为较低。只有在其他控制任务阻塞时，才会执行显示任务。其任务逻辑如下图所示。



2.3 传感器数据处理任务

该任务主要负责接收解析激光测距模块、雷达模块的数据。其触发机制如下：空闲时该任务是阻塞状态，不会运行。当测距模块或雷达模块数据到来时，由 DMA 将数据放入队列中，当队列中存在数据时，调度器会唤醒该任务进行数据处理与解析。解析后的数据将会提供给控制任务使用。

任务名称为 `voidSensorHandleTask(void*param)`，其逻辑如下图所示。



其中 STP23L 激光测距模块解析数据的过程在 `bsp_stp23L.c` 文件内。雷达解析数据过程在 `bsp_stl06n.c` 文件内。

2.4 平衡控制任务

该任务是四轴飞行器控制的核心部分，内部包含了陀螺仪数据的获取与处理、高度数据的使用与处理、雷达数据使用与处理、LQR 控制器、四轴飞行器的状态设置与获取等内容。

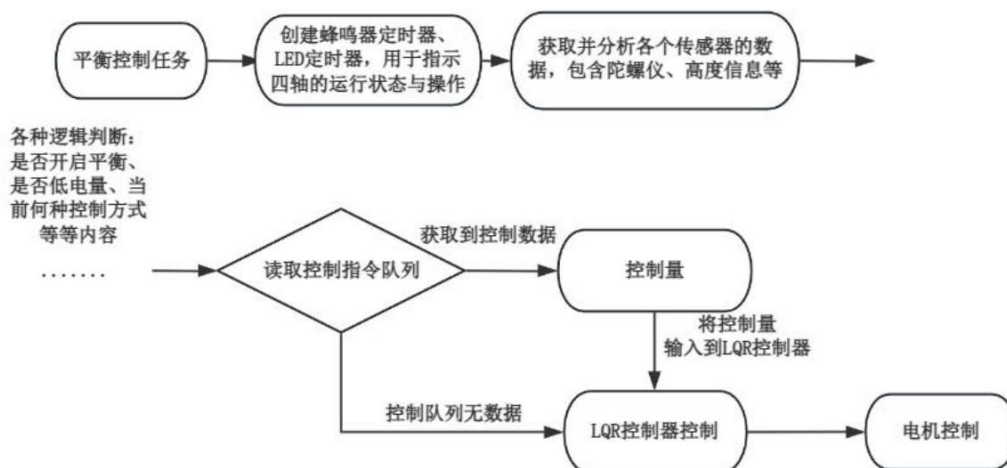
值得注意的是，该任务为主要的核心部分，以 200Hz 的频率运行。四轴的平稳控制需要保证该任务频率足够准确。在设计其他模块或任务时，均需要考虑任务优先级以及某些任务是否长时间占用 CPU 时间等问题。同时该任务内也加入了一个外部的频率检测器，可以用过该检测器来检测任务频率是否准确，其内容如下所示：

```
/*统计任务运行的运行频率*/
g_readonly_BalanceTaskFreq=debug->UpdateFreq(&debugPriv);
```

如果加了其他 DIY 传感器或者功能，可以通过监测

`g_readonly_BalanceTaskFreq` 变量的输出来检查当前的任务频率是否满足要求，正常该值输出为 200，单位是 Hz。若为其他数值，则说明控制任务的频率受到了干扰。

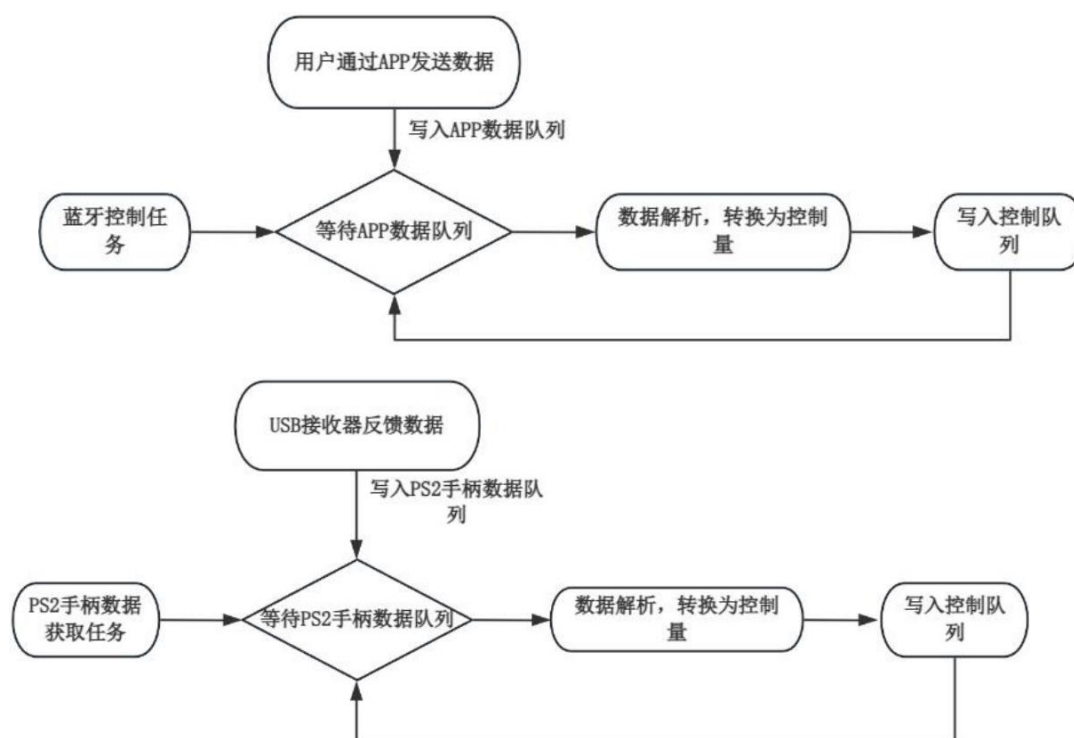
控制任务的名称为 `voidbalance_task(void*param)`，其运行逻辑如下所示：



2.5 APP、PS2 控制任务

四轴飞行器支持 APP 或者 PS2 手柄进行控制，两者的控制方式均是对控制队列写入控制命令。其中控制方式带有优先级，PS2 手柄控制的优先级高于 APP 控制优先级。当 PS2 手柄在执行控制时，APP 控制将被强行占用取消。

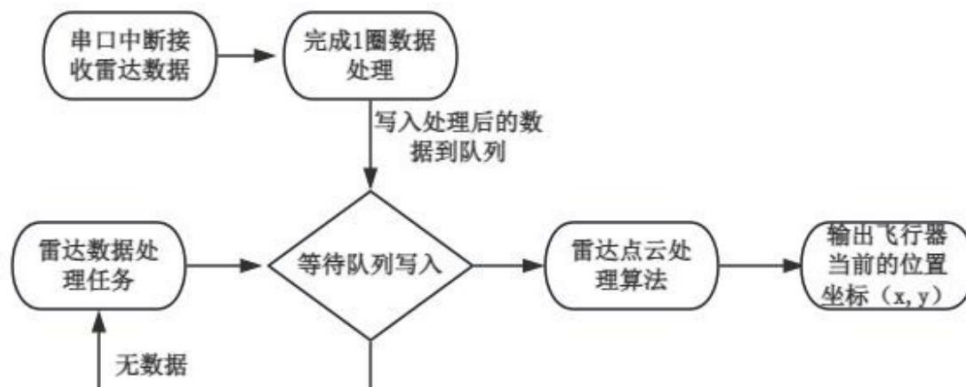
两者的数据底层逻辑不相同，但最终用于控制飞行器动作的方式一致。APP 控制任务名称为 `voidBluetoothAPPControl_task(void*param)`，PS2 控制任务名称为 `voidUSBH_HID_EventCallback(USBH_HandleTypeDef*phost)`。说明：PS2 控制任务在一个 PS2 回调函数中执行，该回调函数在一个 PS2 的数据获取任务内触发。



2.6 雷达数据处理任务

在传感器数据处理任务中，会判断是否完成 1 圈雷达数据的接收。在完成 1 圈雷达数据接收后，将会把数据写入队列，让雷达数据处理任务运行，雷达数据处理任务的内容是将雷达的数据通过算法处理成飞行器的坐标数据。

雷达接收的任务名称为 `voidSensorHandleTask(void*param)`，雷达点云处理算法任务名称为 `voidCoordinateHandleTask(void*param)`。

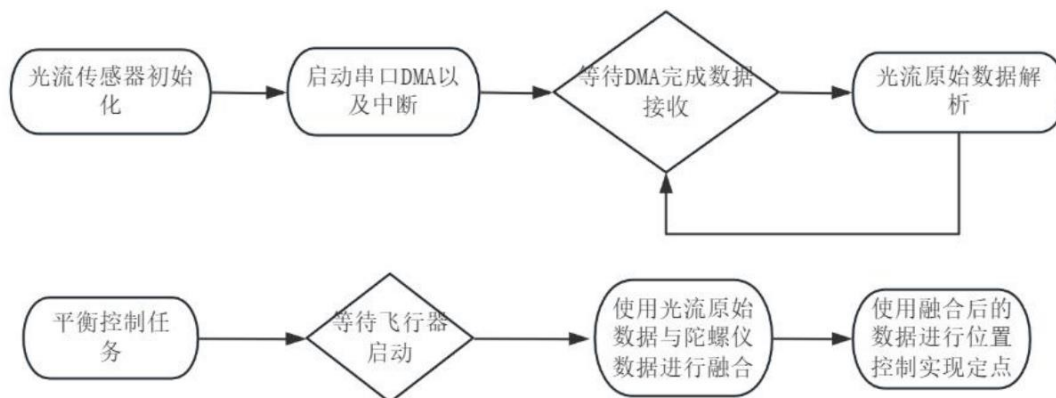


`CoordinateHandleTask`，避障功能，具体表现为：雷达的前后左右四个方向，有一个方向有障碍物靠近时，飞行器就会往相反的方向运动，并发出蜂鸣器提示。此功能为实验性功能，后续可以根据现有内容继续优化，已完成更好的避障效果。

2.7 光流数据处理中断

光流传感器的原始数据直接在串口中断内进行解析，频率 50Hz。具体过程为：光流完成初始化后，将通过一定的格式主动上报数据，每一帧数据上报完成后，就会进入串口空闲中断，在此中断内解析处理 DMA 搬运的数据。

拿到原始数据后，在平衡控制任务将原始数据与陀螺仪数据进行融合增加稳定性，最终把融合后的数据用于位置控制。

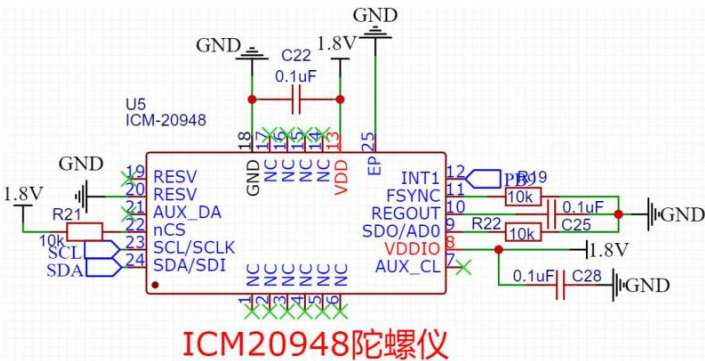


3.硬件与软件实现细节

3.1 陀螺仪

1 陀螺仪电路设计

陀螺仪是四轴飞行器的核心传感器，本产品采用 InvenSense 公司的 ICM20948，原理图如图所示。其中 SCL、SDA 连接至硬件 IIC1 接口，中断输出引脚 INT1 连接到普通的 IO。在本产品中未使用陀螺仪的中断功能，仅获取其原



始数据使用。

2 陀螺仪数据获取

ICM20948 最大支持 400kHz 的 IIC 通信速率，故 IIC 总线做如下图配置，更

Master Features	
I2C Speed Mode	Fast Mode
I2C Clock Speed (Hz)	400000
Fast Mode Duty Cycle	Duty cycle Tlow/Thigh = 2

多细节可自行查看 ioc 配置文件：

为了方便移植以及使用模拟 iic，在软件中对 iic 接口做了以下的封装，在该封装下可以轻松切换软件 iic 以及硬件 iic：

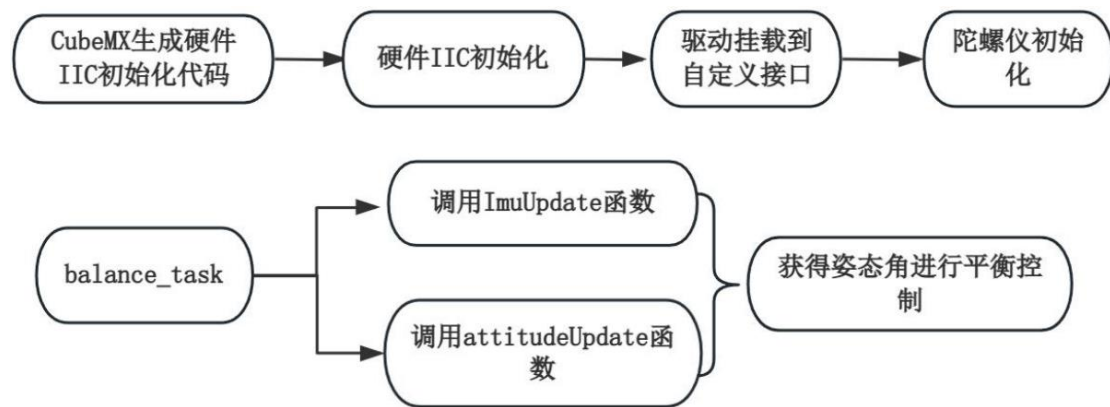
```
typedef struct {  
    //参数：设备地址，要写入的数据，要写入的数据长度，超时时间  
    IIC_Status_t(*write)(uint16_t DevAddress, uint8_t *pData, uint16_t Size,  
        uint32_t Timeout);  
    IIC_Status_t(*read)(uint16_t DevAddress, uint8_t *pData, uint16_t Size,  
        uint32_t Timeout);  
    //参数：设备地址，要访问的寄存器地址，要写入的数据，要写入的数据长  
    度，超时时间
```


硬件初始化完毕 iic 后，再将硬件 iic 的接口函数挂载到上述自定义接口中，即可完成 iic 的所有初始化步骤。至此已经可以顺利使用 iic 与陀螺仪进行通信了，初始化部分可在 `bsp_imc.c` 文件内查看函数 `static uint8_t ICM20948_Init(void)`。

完成 imu 初始化后，后续主要通过两个函数来实现 imu 的原始数据以及姿态解算：

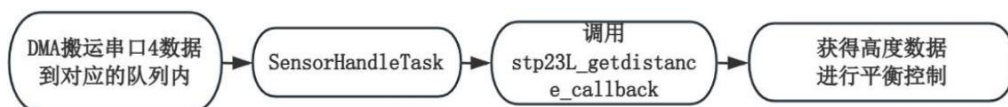
`static void ImuUpdate(IMU_DATA_t*data)`：获取 IMU 的 9 轴原始数值

`static void attitudeUpdate(IMU_DATA_t imuData, ATTITUDE_DATA_t*attitude)`：使用 imu 的原始值进行姿态解算，获得欧拉角。该函数使用显示互补滤波算法，内部的实现细节可查看《四轴无人机开发手册-原理与算法部分》文档。



3.2 激光测距

激光测距模块使用 STP23-L，该传感器的详情在资料文件夹《传感器手册》中可查看。STP23-L 通过串口输出数据，波特率为 230400，数据以点云的形式输出。1 个点云数据内容包含测量距离、信号强度、置信度等信息。其输出频率为 10Hz，1 帧数据内包括 12 个点云信息。



高度数据通过串口接收，最终通过 `bsp_stp23L.c` 文件内的 `stp23L_getdistance_callback` 进行接收以及数据解码。其接收过程如下图所示：

3.3 电量检测

电量检测主要通过测量电池的输入电压来实现。本产品使用磷酸铁锂电池电池，标称电压 12.6V，截至电压 10V。在飞行器运行的过程中，由于飞行器供电线长的关系(2 米)，实测压降为 0.8V 左右。例如飞行器电池电压有 10.8V，在飞行器运行期间，能够输入到飞行器内的电压为 $10.8 - 0.8 = 10V$ 。为了保证有足

够的电量进行飞行，在程序内部设定了电量低的保护，当电池电压低于 10.8V 时，将不允许启动。

1 电压测量电路实现

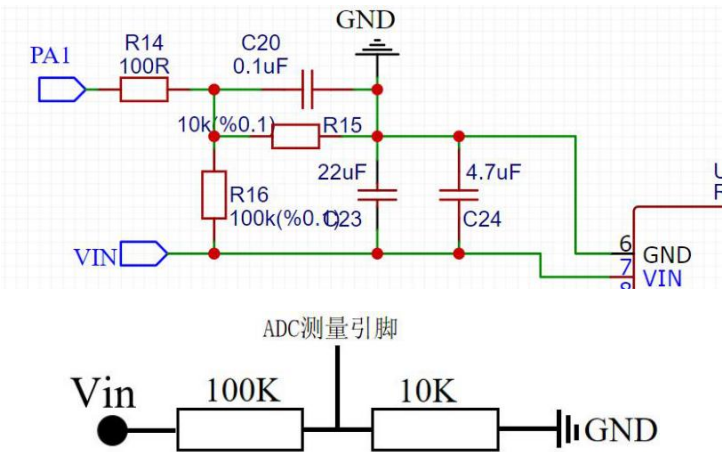
在电池输入处，串联了一个 100k 和 10k 的电阻实现分压，并使用 ADC 相关的引脚进行模拟量采集，此处为 PA1，如下图所示。

根据简单的分压公式可得知，测量点处的电压为： $V_{in}=V_{ADC} \times 11$

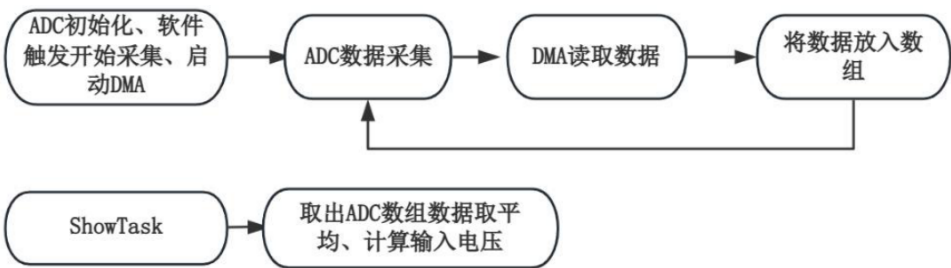
ADC 测量引脚电压范围在 0~3.3V，对应 ADC 采样值 0~4095，可得出最终的输入电压公式为： $V_{in}=ADC \text{ 采样原始值}/4095 \times 3.3 \times 11$

2 程序实现

为确保不影响系统的实时性，ADC 配置为软件触发、连续采集，并使用 DMA



循环搬运数据，不占用 CPU 资源。为保证采样的准确性，采用时配置了最大的采样周期，同时让 DMA 搬运到一个长度为 80 的数组中，最终取平均值计算。具体的配置可查看 ioc 配置文件，程序的实现在 bsp_adc.c 文件内。最终的输入



电压计算由显示任务完成，其处理的内容主要是在数组中取出数据取平均，然后根据上述提到的公式计算出输入电压。

3.4 电机控制

1 硬件部分

电机控制的硬件部分主要是引出 TIM8 的 4 个通道引脚，作为一个接口与电

调连接，如下图所示。飞控与电调的连接如下：

PC6---E1（E1~E4 为电机控制信号线，ADC 用于电调电流采集）

PC7---E2

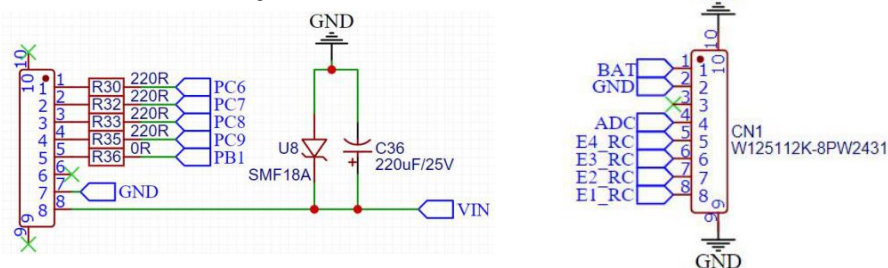
PC8---E3

PC9---E4

PB1---ADC

电调负责的工作：将主控芯片发过来的信号解析，最终转换为油门的大小，控制无刷电机旋转。

飞控板的工作：将 LQR 控制器输出的信号转化为油门值，通过 PWM 引脚

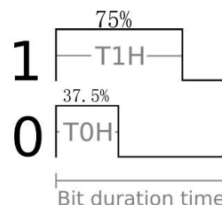


发送到电调。

飞控板与电调通信的协议为 Dshot300。下面简述 Dshot 协议的一些基本内容：目前 Dshot 协议常见类型有 Dshot150、Dshot300、Dshot600 和 Dshot1200。后面的数字代表在通信过程中，每秒传输多少 kbit，类似于串口的波特率。例如本产品中使用 Dshot300 协议，代表飞控每秒可传输 300kbit 数据到电调。

在 Dshot 协议中，一个周期脉冲表示一个 bit，发送的数据位 0 或者 1 则通过一个周期内的占空比来表示。所以 Dshot 与 PWM 有如下的关系：Dshot 协议的速度类型决定 PWM 的频率，Dshot 发送的数据位 0 或者 1 决定 PWM 的占空比。

Dshot 协议中的 0 或者 1 的占空比对应关系如下图：



逻辑 1 对应 PWM 占空比 75%

逻辑 0 对应 PWM 占空比 37.5%

Dshot 协议在通信时，一帧数据共有 16 个 bit 构成，其中前 11 位是油门信号，可表示的范围是 0~2047。第 12 位是命令请求或者命令设置使能位，后 4 位

是校验位。

油门信号：在油门值中，0~47 的数值范围用于一些命令的设置，比如设置电调的 LED 指示灯、设置电机的旋转方向等功能。所以真实的油门值信号从 48 开始，48 表示最低油门，电机不会旋转；2047 表示满油门，电机将会以最大速度运行。

命令请求位/设置使能位：如果需要对电调进行设置时，例如上述提到的修改电机旋转方向或 LED 指示灯，则需要把这个位置 1。若需要对电机进行控制（设置 48~2047 油门值），需要将此位设置为 0。

校验位：校验位是将前 12 位数据分为 3 组，每组 4bit，进行异或校验。假设当前有一个 12 位的 SendData 数值需要校验，则校验方式为：

$$csum = (SendData \wedge (SendData \gg 4) \wedge (SendData \gg 8)) \& 0xf$$

下面一帧完整的 dshot 数据演示，假设当前需要设置油门值

1046(10000010110b), 则命令使能位应该位 0，则可计算到校验位为 0x06(0110b), 数据帧如下图所示：

更多 Dshot 内容,推荐参考网址: [DSHOTintheDark-dmrlawson](https://www.dshotintheDark.com/)

在简单了解上述 Dshot 的通信协议与原理后，就可以对飞控板进行定时器的配置了。首先是 PWM 的配置频率，这里使用的是 Dshot300 协议，速率为 300kbit/s，也就是 PWM 每秒输出 300k 个脉冲，故 PWM 配置为 300000Hz。

飞控板使用的是 TIM8 作为通信的定时器，其主频为 168MHz，为了方便计算，将定时器的预分频系数配置为 0。可算的自动重装载值配置应为 559。

$$PWM \text{ 频率} = \frac{168M}{(0+1) \times (559+1)} = 300kHz$$

将 TIM8 配置为 PWM1 输出，向上计数。要实现发送 dshot 的 0 和 1 指令时，只需要修改定时器的 CCRx 寄存器即可。当需要发送逻辑 1 时，可通过配置对应通道的 CCRx 寄存器输出 75% 的占空比；需要发送逻辑 0 时，配置对应通道的 CCRx 寄存器输出 37.5 占空比。

$$\text{一个周期内 PWM 占空比} = \frac{\text{CCRx 寄存器值}}{560 \text{ (自动重装载值)}}$$

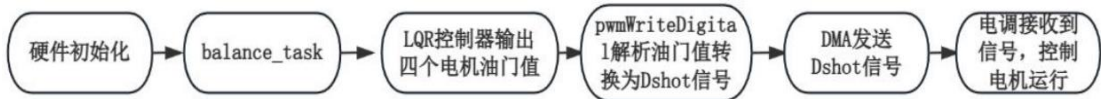
根据上述的配置，已经可以正确的向电调发送 0 和 1 的数据位了，此时需要一个函数，直接将需要发送的油门值进行解析，然后通过 PWM 的形式发送到电调中。本产品使用 DMA 的方式，将解析后的数据搬运到定时器对应通道的 CCRx 寄存器内，实现与电调通信，具体内容可在 `bsp_dshot.c` 文件内的 `pwmWriteDigital` 函数查看。其核心部分是将 0 与 1 通过 CCRx 寄存器的值表示，以及通过 DMA

搬运的方式使得 PWM 能准确输出对应占空比，节选内容如下：
整体的电机控制流程图如下图

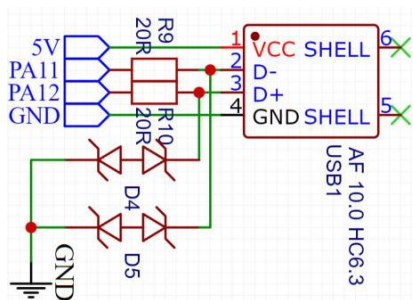
3.5PS2 手柄控制

```
uint8_t i=0;
for(i=0;i<16;i++)
{
    g_esc_cmd[0][i] = ((m1.throttle>>(15-i))&0x01) ? ESC_BIT_1 : ESC_BIT_0;
    g_esc_cmd[1][i] = ((m2.throttle>>(15-i))&0x01) ? ESC_BIT_1 : ESC_BIT_0;
    g_esc_cmd[2][i] = ((m3.throttle>>(15-i))&0x01) ? ESC_BIT_1 : ESC_BIT_0;
    g_esc_cmd[3][i] = ((m4.throttle>>(15-i))&0x01) ? ESC_BIT_1 : ESC_BIT_0;
}
#if NOT_DMA_MOTOR //非 DMA 方式
while(1) if( ESC_CMD_BUF_LEN == g_esc_index ) break;
g_esc_index = 0;
HAL_TIM_Base_Start_IT(DshotTIM);
#else //DMA
HAL_TIM_PWM_Start_DMA(DshotTIM, TIM_CHANNEL_1, (uint32_t*)&g_esc_cmd[0][0], ESC_CMD_BUF_LEN);
HAL_TIM_PWM_Start_DMA(DshotTIM, TIM_CHANNEL_2, (uint32_t*)&g_esc_cmd[1][0], ESC_CMD_BUF_LEN);
HAL_TIM_PWM_Start_DMA(DshotTIM, TIM_CHANNEL_3, (uint32_t*)&g_esc_cmd[2][0], ESC_CMD_BUF_LEN);
HAL_TIM_PWM_Start_DMA(DshotTIM, TIM_CHANNEL_4, (uint32_t*)&g_esc_cmd[3][0], ESC_CMD_BUF_LEN);
#endif
#endif
```

1 硬件部分

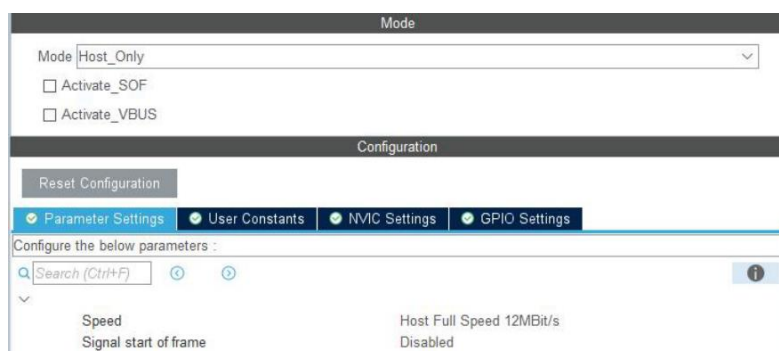


手柄采用 USB 通信，飞控板作为主机 USBHost，PS2 手柄接收器作为从机。
电路如下图：

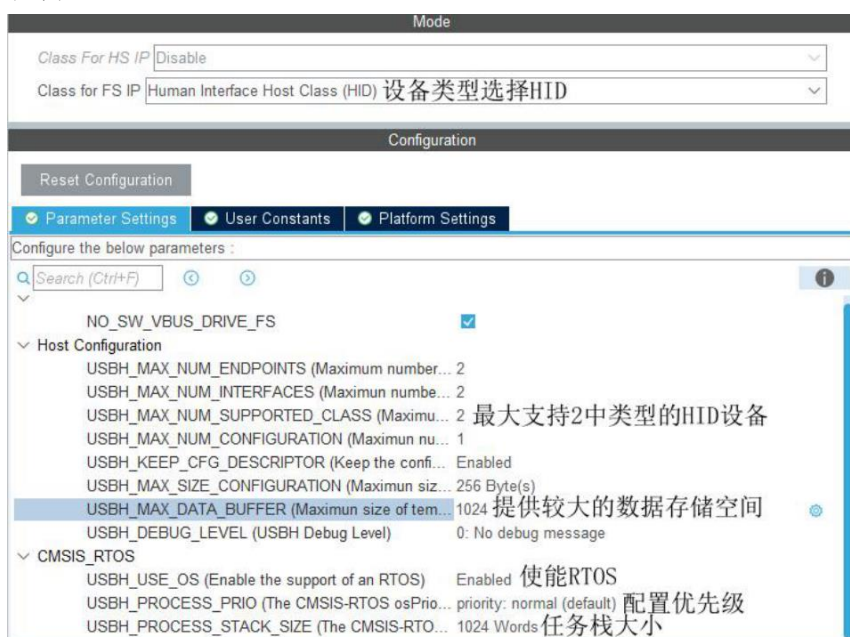


2 软件部分

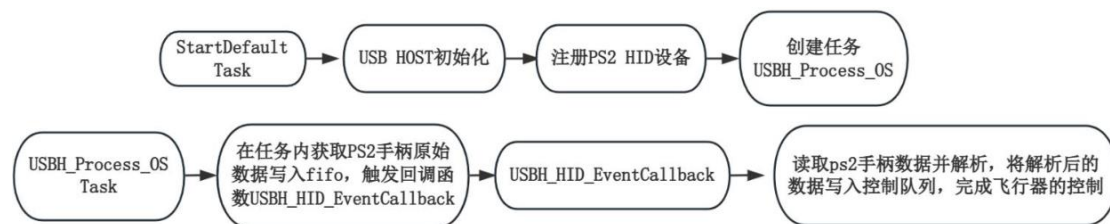
在 CubeMX 中，将 USB_OTG_FS 全速设备使能，如下图所示：



使能 USBOTG 后，在中间件里面配置 USB 设备的类型，手柄类型是 HID。本产品使用的无线 PS2 手柄内置两种模式，一种为 PC 模式，一种 Android 模式，两种模式是不同的 HID 设备类型，所以在最大支持的 HID 设备类中，配置为 2。另外 PS2 设备输出的数据长度较长，需要给存放数据的 Buffer 配置较大长度，这里为 1024bytes。此外使用到了 FreeRTOS，所以在 RTOS 配置栏需要配置优先级，以及任务的栈大小，如下图所示。



完成 CubeMX 配置后，需要手动创建 PS2 的 HID 类，并提供接口初始化函数、数据请求函数、数据解析函数等内容，此部分内容可参考官方提供的 `usbh_hid_keybd.c` 或 `usbh_hid_mouse.c` 学习。PS2 手柄相关的细节位于 `bsp_ps2.c` 文件内。在创建完 PS2 手柄的 HID 类后，需要将设备注册到 USB HOST，注册以及初始化过程均在 `usb_host.c` 文件 `MX_USB_HOST_Init` 函数完成。该函数在默认任务中调用，整体的流程如下图所示：



完成上述的过程，PS2 手柄数据将会被存放在一个结构体内：

```

typedef struct{
    uint8_t LX; //4个方向摇杆值,取值0~255
    uint8_t LY;
    uint8_t RX;
    uint8_t RY;
    PS2KEY_State_t (*getKeyEvent)(uint8_t keybit);
    uint8_t (*getKeyState)(uint8_t keybit);}PS2INFO_t;
  
```

通过这个结构体可以方便的使用 PS2 手柄数据：LX、LY:表示左摇杆的左右、上下幅度，范围是 0~255 RX、RY:表示右摇杆的左右、上下幅度，范围是 0~255 如果想获取按键的事件，包含单击、双击、长按事件，可以通过 `getKeyEvent` 方法获取；如果想获取按键的状态，被按下或者未按下，可通过 `getKeyState` 方法获取。具体可参考 `ps2_task.c` 文件下的 `USBH_HID_EventCallback` 内容。

示例如下：

```

extern PS2INFO_t ps2_info;
PS2INFO_t* ps2 = &ps2_info; //获取 ps2 手柄信息指针

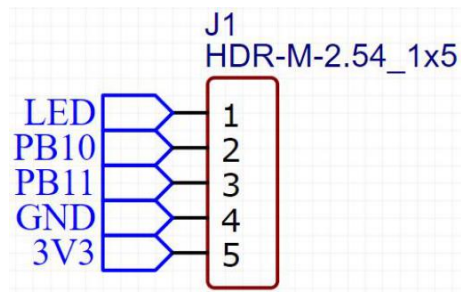
//获取 PS2 手柄“Start”按键的事件
ps2_keystate = ps2->getKeyEvent(PS2KEY_START);
if( PS2KEYSTATE_LONGCLICK == ps2_keystate ) //长按事件
{
}
else if( PS2KEYSTATE_SINGLECLICK == ps2_keystate ) //单击事件
{
}
else if( PS2KEYSTATE_DOUBLECLICK == ps2_keystate ) //双击事件
{
}
  
```

其中 `getKeyEvent` 和 `getKeyState` 的传入参数为 PS2 手柄上 16 个按键的枚举值，可以在 `bsp_ps2.h` 文件中查看。

3.6 APP 控制

1 硬件部分

APP 通信采用的是蓝牙透传的方式，通过串口 3 与蓝牙模块连接，完成数据透传。电路如下图所示：



2 软件部分

蓝牙模块的出厂波特率为 230400，所以在 CubeMX 中对串口 3 做对应的初始化内容即可。在串口接收到手机 APP 数据时，会进入到串口的接收中断回调函数，并将数据写入到队列中。队列收到数据后，调度器将唤醒 `BluetoothAPPControl_task` 任务进行 APP 数据的解析与处理。

